This mechanism is another example of how one behavior is composed with another, like matryoshka dolls: the supervisor is nothing more than a wrapping behavior on top of the actor's behavior. It intercepts all messages sent to the actor and wraps them in a `try`/`catch` before passing them on, which allows the actor's exceptions to be intercepted. This approach not only takes care of its exceptions but also overrides the return behavior of its inner actor: in figure 5.7, the `Behavior.same` in step 8.

Overriding the actor's next behavior is necessary because if an exception is thrown within the actor, execution terminates at that point, and the original next behavior of the actor can no longer be reached. You can supervise `someBehavior` using `Behaviors.supervise(someBehavior)`.

Then you need to add the supervision strategy to use with `.onFailure`. If you want the actor to maintain the same state it had before the exception was thrown, you must use `resume`. If you use `restart` instead, the actor loses its previous state.

The following snippet is an example of how to use the resume strategy:

```
val behavior: Behavior[Command] = ...                                    ❶

Behaviors.supervise(behavior).onFailure(SupervisorStrategy.resume)       ❷
```

❶ Actor's behavior

❷ Supervision strategy

In this setting, every exception is treated the same. Any throwable exception that is not fatal is handled with the resume strategy.